



Green University of Bangladesh

Department of Computer Science and Engineering (CSE)

Faculty of Sciences and Engineering

Semester: (Summer, Year: 2026), B.Sc. in CSE (Day)

Open-Ended Laboratory Report

Course Title : Computer Network

Course Code : CSE-318 Section: 241-D2

Project / Experiment Title : Multi-Client Examination System Using TCP

Student Details

No.	Name	Student ID
1	Md. Shajalal	223002088

Open-Ended Lab Date : 16-08-26

Submission Date : 16-08-26

Course Teacher's Name : Mahbubur Rahman

Open-Ended Laboratory Assessment Status

Marks: _____	Signature: _____
Comments: _____	Date: _____

1 Title

Multi-Client Examination System Using TCP

2 Objectives and Problem Statement

2.1 Objectives

The main objective of this project is to develop an online examination system that allows multiple students to take an examination simultaneously. The system is divided into client-side and server-side responsibilities.

2.2 Client-Side Objectives

The client will provide the following functionalities:

- **Login:** Allow students to securely log in to the examination system.
- **Start Exam:** Allow authenticated students to start the examination.
- **Answer Questions:** Display questions and allow students to select or submit their answers.
- **Submit Exam:** Allow students to submit their completed examination.
- **View Score:** Display the student's examination score after submission.

2.3 Server-Side Objectives

The server will be responsible for the following functionalities:

- **Authentication:** Verify student credentials and manage secure access to the system.
- **Send Questions:** Provide examination questions to authenticated students.
- **Receive Answers:** Receive and process answers submitted by students.
- **Calculate Score:** Evaluate submitted answers and calculate the student's score.
- **Return Result:** Send the calculated examination result back to the client.

2.4 Concurrent Examination

The system will support multiple students taking the examination simultaneously. The server will handle requests from multiple clients concurrently while maintaining the accuracy and integrity of each student's examination session and result.

2.5 Problem Statement

A single-client examination application cannot efficiently handle multiple students taking an examination simultaneously. This creates a need for a network-based examination system that can support multiple concurrent clients while maintaining independent communication and examination results for each student. The proposed system uses TCP socket programming for client-server communication and multithreading to handle multiple students concurrently.

3 Proposed Solution Design

3.1 Relevant Networking Concepts and Problem Analysis

This project uses client-server communication, TCP sockets, and multithreading. The student connects to the server using a socket, sends login details and answers, and receives questions and results from the server. The main problem is to allow multiple students to take the exam at the same time. To solve this, the server creates a separate thread for each student. This makes the system simple and allows multiple clients to communicate with the server at the same time.

3.2 Selected Method and Justification

We selected TCP socket programming with multithreading for this project. TCP provides reliable communication between the student client and the exam server. The client and server can send and receive information, such as login details, questions, answers, and results.

For multiple students, the server creates a separate thread for each client. This allows several students to take the exam at the same time. This method is suitable for our project because it is simple, reliable, and directly uses the socket programming and multithreading concepts learned in the networking lab.

3.3 Main Solution Steps

1. Start the server and wait for student connections.
2. Connect the student client to the server using TCP socket.
3. Login and authenticate the student using username and password.
4. Conduct the exam by sending questions and receiving answers.
5. Calculate and send the result after the student submits the exam.

3.4 Communication Algorithm / Pseudocode / Flowchart

3.5 Communication Algorithm

1. Start the server and create a `ServerSocket`.
2. Wait for students to connect.
3. Accept a client connection.
4. Create a separate thread for the connected student.
5. Client sends username and password to the server.
6. Server checks the login information and sends the login result.
7. If login is successful, the server sends the exam questions.
8. Client displays each question and sends the student's answer to the server.
9. Server receives the answers and continues sending the remaining questions.
10. Client sends `SUBMIT` after completing the exam.
11. Server calculates the score.

12. Server sends the score back to the client.
13. Client displays the result and closes the connection.
14. The server continues waiting for other students.

3.6 Flowchart

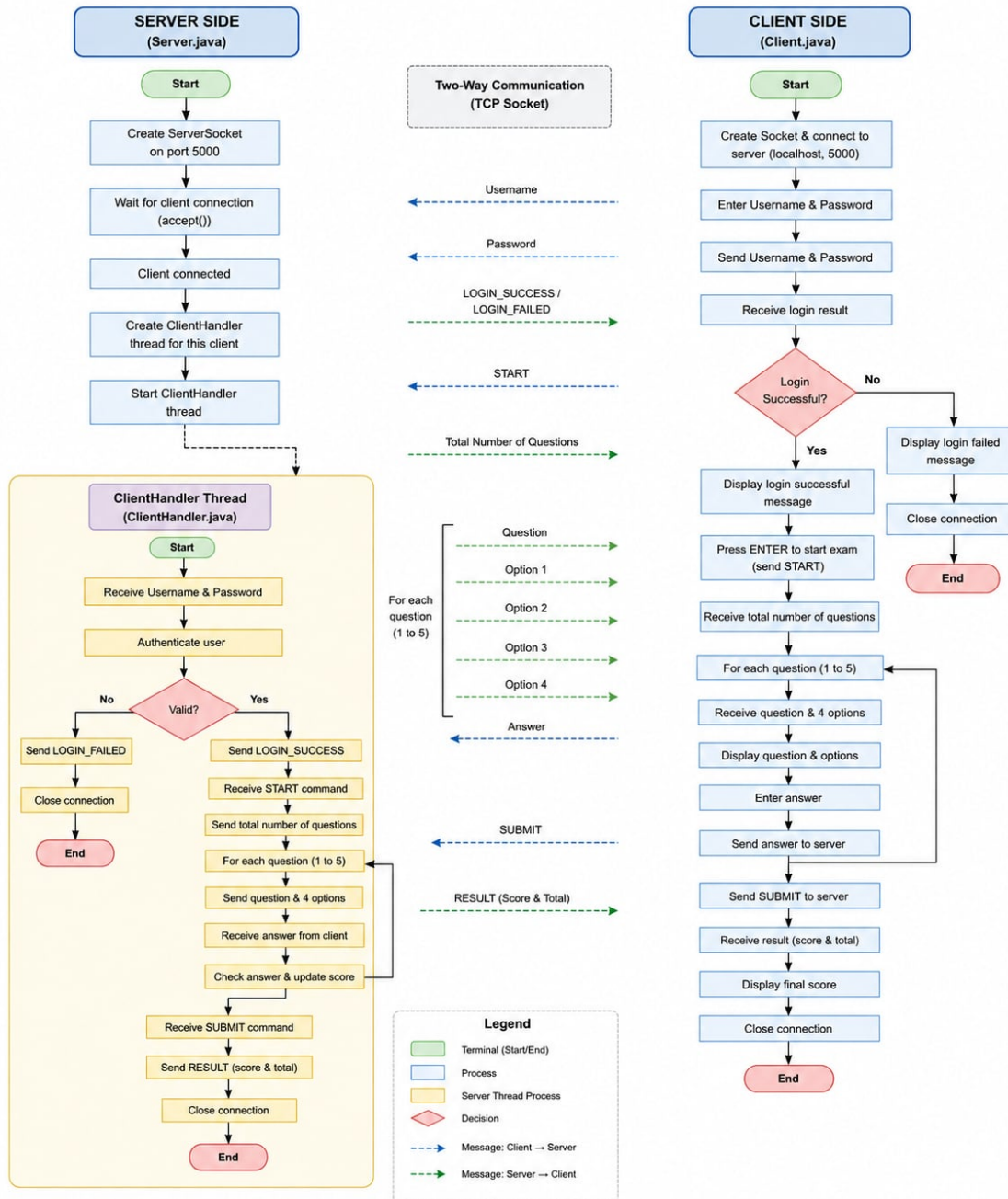


Figure 1: Proposed solution design

4 Implementation and Technical Details

- **Programming Language:** Java
- **Communication Protocol:** TCP
- **Server:** ServerSocket
- **Client:** Socket

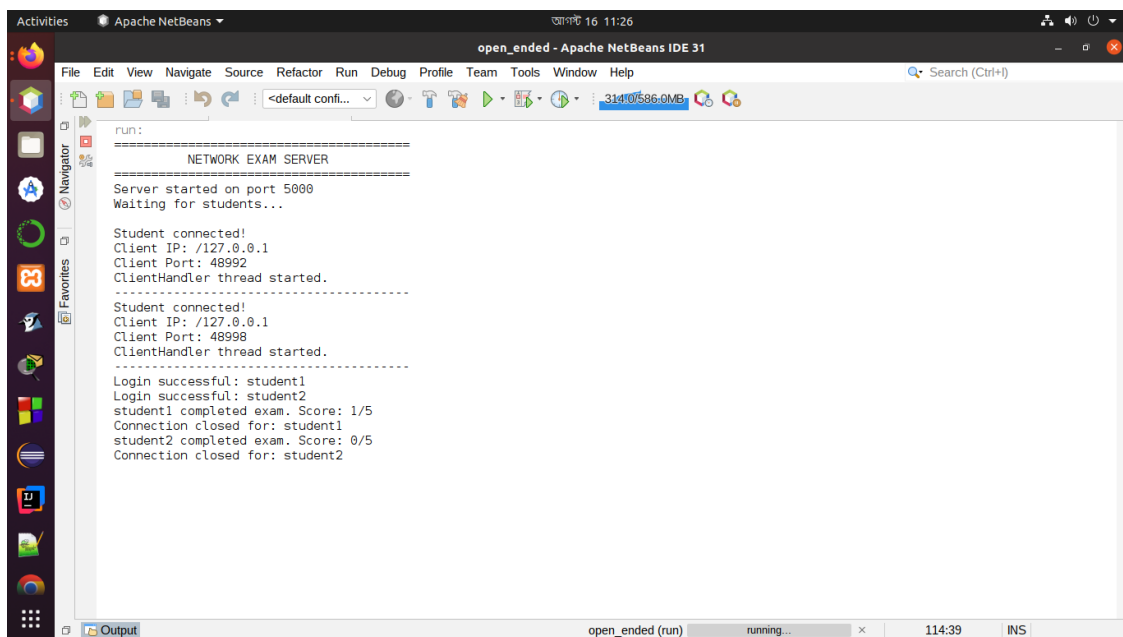
- **Data Exchange:** Input and Output Streams
- **Concurrent Clients:** Java Threads
- **Server Thread:** ClientHandler
- **Main Functions:** Login, Start Exam, Answer Questions, Submit, and View Score

This implementation is kept simple and focuses on the main networking concepts of **socket communication**, **two-way data exchange**, and **multithreading**.

5 Result Analysis and Discussion

After testing the system, we found that the main features were working properly. Students could log in, receive the exam questions, submit their answers, and get their scores from the server. We also checked multiple clients, and the server was able to handle them using separate threads. Overall, the system worked well for our lab project and helped us understand how socket communication and multithreading work in a real application.

5.1 Output Evidence



```

run:
=====
      NETWORK EXAM SERVER
=====
Server started on port 5000
Waiting for students...

Student connected!
Client IP: /127.0.0.1
Client Port: 48992
ClientHandler thread started.
-----
Student connected!
Client IP: /127.0.0.1
Client Port: 48998
ClientHandler thread started.
-----
Login successful: student1
Login successful: student2
student1 completed exam. Score: 1/5
Connection closed for: student1
student2 completed exam. Score: 0/5
Connection closed for: student2
  
```

Figure 2: Output of Test Case 1

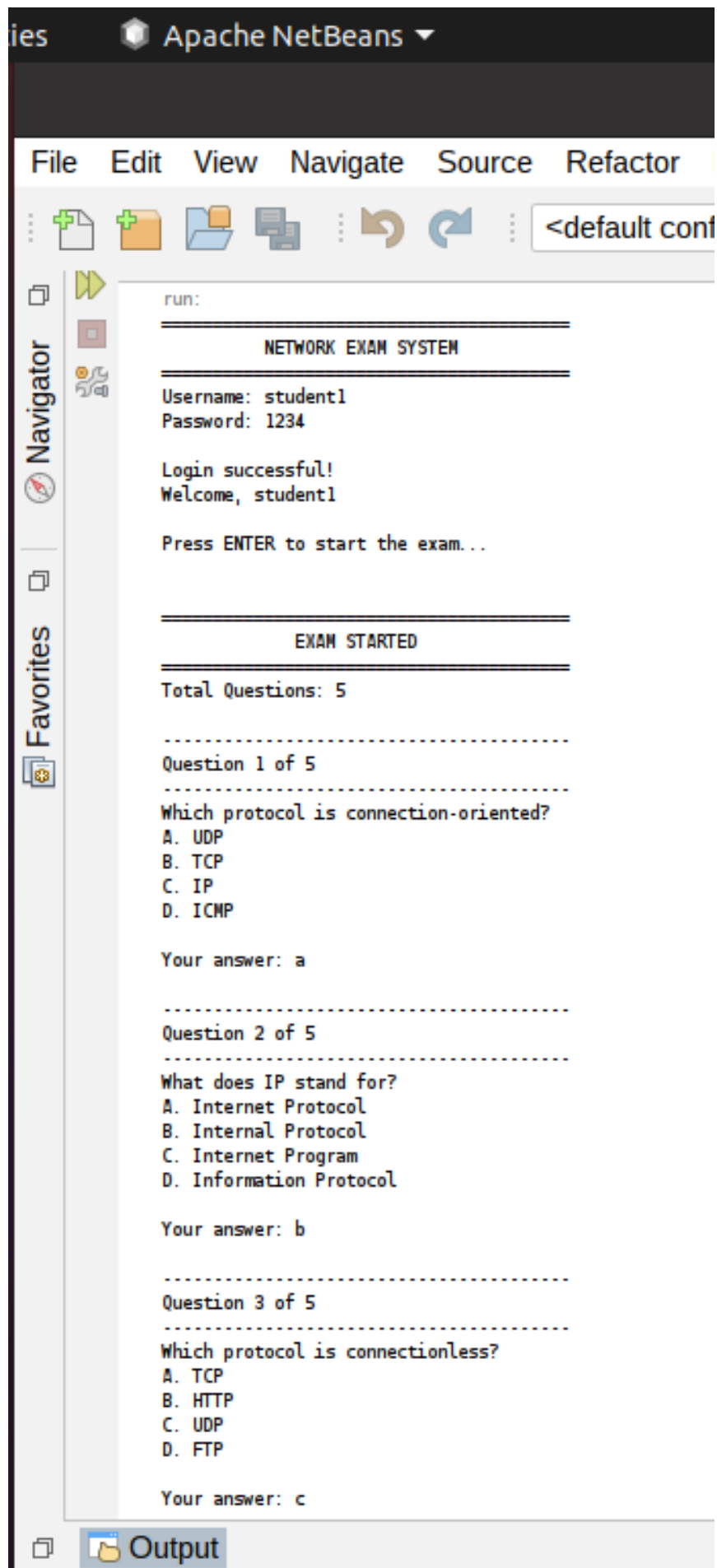


Figure 3: Output of Test Case 2

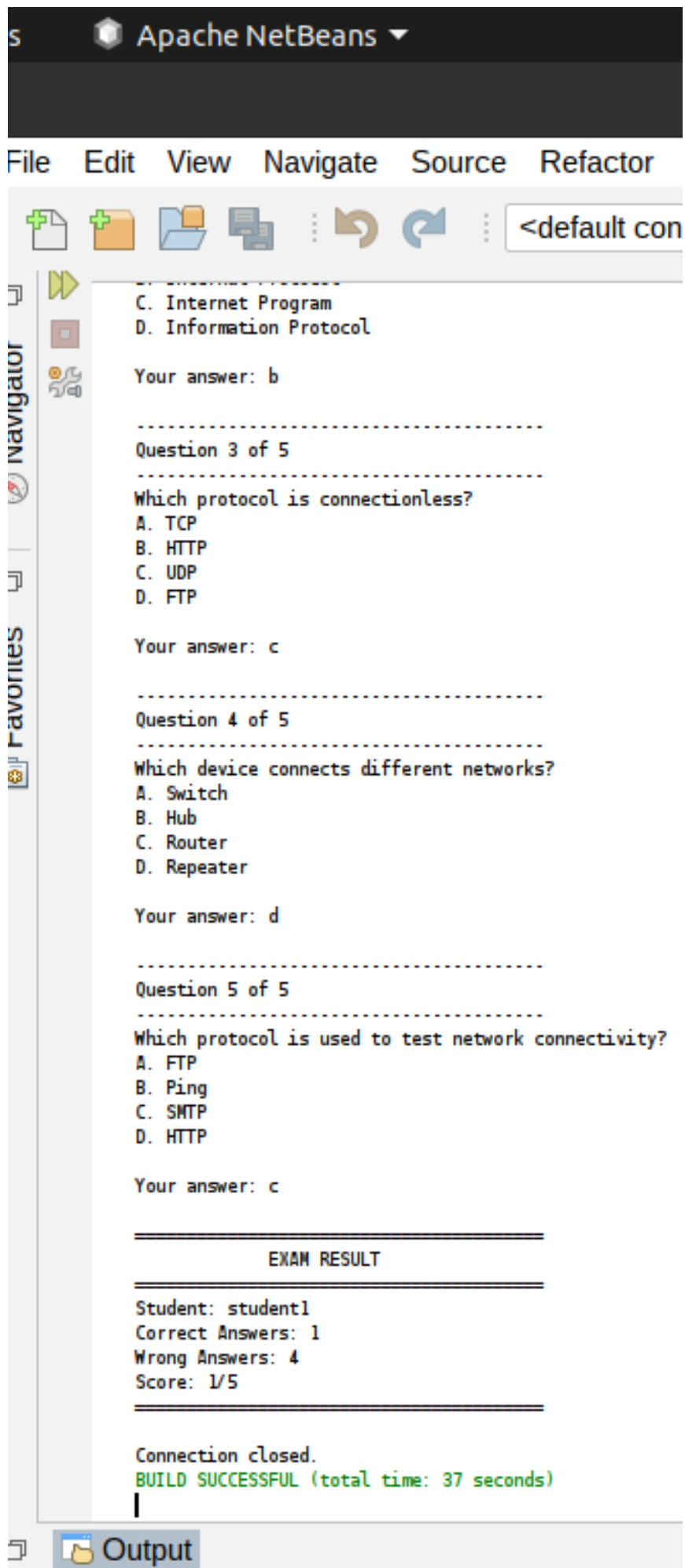


Figure 4: Output of Test Case 3

5.2 Analysis, Observation, and Challenges

5.2.1 Analysis

We tested the system by connecting students to the server. The main functions, such as login, receiving questions, submitting answers, and getting the score, worked properly. The server was also able to handle multiple students using separate threads.

5.2.2 Observation

During testing, we observed that the client and server could communicate correctly. The server sent the questions to the students and received their answers. We also observed that each student could take the exam separately without affecting other students.

5.2.3 Challenges

During the project, we faced some difficulties in connecting the client and server and maintaining the correct order of communication. Handling multiple students with threads was also a challenge. We also needed to make sure that login information, answers, and scores were handled correctly for each student.

6 Conclusion, Limitations and Future Improvement

6.1 Conclusion

In this project, we developed a simple Network-Based Online Examination System using Java socket programming and multithreading. The main purpose was to allow students to connect to a server, log in, take an exam, submit their answers, and receive their scores.

The project helped us understand how a client and server communicate with each other over a network. We also learned how multithreading allows the server to handle more than one student at the same time. Overall, the project was useful for applying the networking concepts we learned in the lab to a practical problem.

6.2 Limitations and Future Improvement

6.2.1 Limitations

The system has some limitations. The student information and questions are stored inside the program, so there is no database. The system also has a simple interface and does not include an exam timer or random questions. It is mainly designed for a lab project and may not be suitable for a large number of students.

6.2.2 Future Improvement

In the future, a database can be added to store students, questions, and results. A better graphical interface can also make the system easier to use. An exam timer and random questions can be

added. The system can also be improved to support more students and provide a teacher or admin section for managing questions and results.

7 References

1. Computer Networking Lab Manual, course materials provided by the instructor.
2. Java documentation for `Socket` and `ServerSocket`.
3. Java documentation for `Threads` and multithreading.
4. Class notes and laboratory exercises on TCP client-server communication.
5. Class notes and laboratory exercises on socket programming and multithreading.